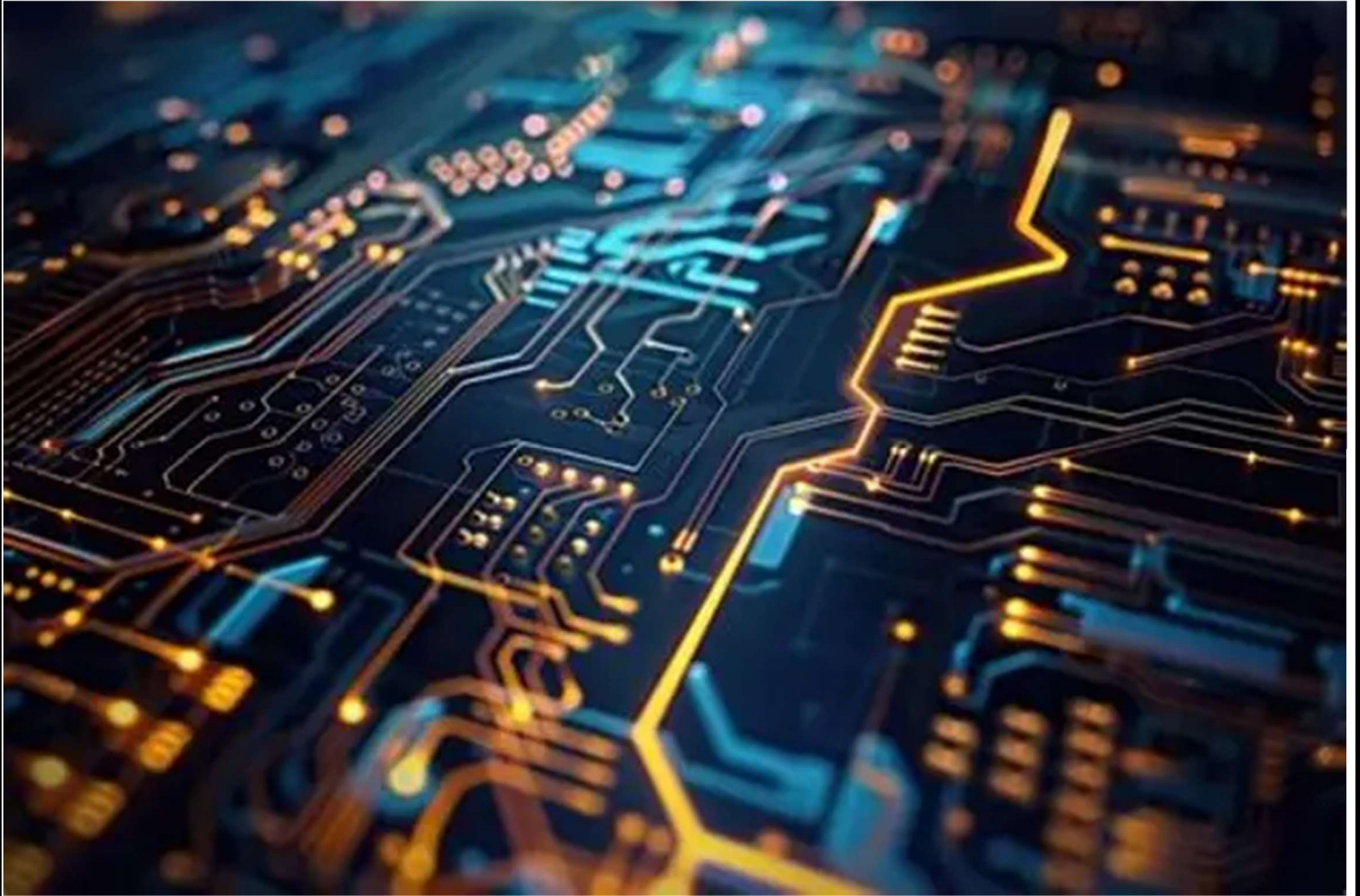


Assignment: Gray to seven segments



Name: Ahmed Wael Sadek El
Sawy

Assignment: Gray to seven segments

Gray to seven segments

gray2bin.v

```
module gray2bin #(
    parameter N = 4
) (
    input  wire [N-1:0] gray,
    output wire [N-1:0] bin
);
    genvar i;

    assign bin[N-1] = gray[N-1];

    generate
        for (i = N-2; i >= 0; i = i - 1) begin : gen_gray_conv
            assign bin[i] = bin[i+1] ^ gray[i];
        end
    endgenerate
endmodule
```

bin2seg.v

```
module bin2seg (
    input  wire [3:0] bin,
    output reg  [6:0] seg
);
    // Active Low Encoding: 0 = ON, 1 = OFF
    always @(*) begin
        case (bin)
            //gfedcba
            4'h0: seg = 7'b1000000;
            4'h1: seg = 7'b1111001;
            4'h2: seg = 7'b0100100;
            4'h3: seg = 7'b0110000;
            4'h4: seg = 7'b0011001;
            4'h5: seg = 7'b0010010;
            4'h6: seg = 7'b0000010;
            4'h7: seg = 7'b1111000;
            4'h8: seg = 7'b0000000;
            4'h9: seg = 7'b0010000;
            4'hA: seg = 7'b0001000;
            4'hB: seg = 7'b0000011;
            4'hC: seg = 7'b1000110;
            4'hD: seg = 7'b0100001;
```

Assignment: Gray to seven segments

```
        4'hE: seg = 7'b00001110;
        4'hF: seg = 7'b00011110;
        default: seg = 7'b11111111;
    endcase
end
endmodule
```

top_gray2seg.v

```
module top_gray2seg (
    input wire [3:0] in_gray,
    output wire [6:0] seg_out
);
    // Internal wire connecting the two sub-modules
    wire [3:0] out_binary;

    gray2bin #(.N(4)) u_gray2bin (
        .gray(in_gray),
        .bin(out_binary)
    );

    bin2seg u_bin2seg (
        .bin(out_binary),
        .seg(seg_out)
    );
endmodule
```

tb_top_gray2seg.v

```
module tb_top_gray2seg;
    // Testbench signals
    reg [3:0] in_gray;
    wire [6:0] seg_out;

    // Self-checking variables
    integer i;
    integer errors = 0;
    reg [3:0] expected_bin;
    reg [6:0] expected_seg;

    // Instantiate Top Module
    top_gray2seg DUT (
        .in_gray(in_gray),
        .seg_out(seg_out)
    );
endmodule
```

Assignment: Gray to seven segments

```
// Helper function to return expected active-low 7-seg values
function [6:0] get_expected_seg;
    input [3:0] bin_val;
    begin
        case (bin_val)
            4'h0: get_expected_seg = 7'b1000000;
            4'h1: get_expected_seg = 7'b1111001;
            4'h2: get_expected_seg = 7'b0100100;
            4'h3: get_expected_seg = 7'b0110000;
            4'h4: get_expected_seg = 7'b0011001;
            4'h5: get_expected_seg = 7'b0010010;
            4'h6: get_expected_seg = 7'b0000010;
            4'h7: get_expected_seg = 7'b1111000;
            4'h8: get_expected_seg = 7'b0000000;
            4'h9: get_expected_seg = 7'b0010000;
            4'hA: get_expected_seg = 7'b0001000;
            4'hB: get_expected_seg = 7'b0000011;
            4'hC: get_expected_seg = 7'b1000110;
            4'hD: get_expected_seg = 7'b0100001;
            4'hE: get_expected_seg = 7'b0000110;
            4'hF: get_expected_seg = 7'b0001110;
            default: get_expected_seg = 7'b1111111;
        endcase
    end
endfunction

initial begin

    // Initialize
    in_gray = 4'b0000;
    #10;

    // Exhaustive test: Loop through all 16 binary numbers
    for (i = 0; i < 16; i = i + 1) begin
        expected_bin = i;

        // Calculate the Gray Code stimulus dynamically:  $G = (B \gg 1) \oplus B$ 
        in_gray = (expected_bin >> 1) ^ expected_bin;

        // Get the expected 7-segment output based on the binary value
        expected_seg = get_expected_seg(expected_bin);

        #10;
    end
end
```

Assignment: Gray to seven segments

```
// Verify output
if (seg_out !== expected_seg) begin
    $display("ERROR: Gray Input %b (Bin %h) | Expected Seg: %b, Got:
%b",
            in_gray, expected_bin, expected_seg, seg_out);
    errors = errors + 1;
end else begin
    $display("PASS: Gray Input %b -> Internal Bin %h -> Seg Out %b",
            in_gray, expected_bin, seg_out);
end
end

if (errors == 0)
    $display("SUCCESS: Top Level Module passed all 16 tests!");
else
    $display("FAILED with %0d errors.", errors);
    $stop;
end
endmodule
```

Testing gray to seven segments

Assignment: Gray to seven segments

```
# Loading work.gray2bin(fast)
# Loading work.bin2seg(fast)
# PASS: Gray Input 0000 -> Internal Bin 0 -> Seg Out 1000000
# PASS: Gray Input 0001 -> Internal Bin 1 -> Seg Out 1111001
# PASS: Gray Input 0011 -> Internal Bin 2 -> Seg Out 0100100
# PASS: Gray Input 0010 -> Internal Bin 3 -> Seg Out 0110000
# PASS: Gray Input 0110 -> Internal Bin 4 -> Seg Out 0011001
# PASS: Gray Input 0111 -> Internal Bin 5 -> Seg Out 0010010
# PASS: Gray Input 0101 -> Internal Bin 6 -> Seg Out 0000010
# PASS: Gray Input 0100 -> Internal Bin 7 -> Seg Out 1111000
# PASS: Gray Input 1100 -> Internal Bin 8 -> Seg Out 0000000
# PASS: Gray Input 1101 -> Internal Bin 9 -> Seg Out 0010000
# PASS: Gray Input 1111 -> Internal Bin a -> Seg Out 0001000
# PASS: Gray Input 1110 -> Internal Bin b -> Seg Out 0000011
# PASS: Gray Input 1010 -> Internal Bin c -> Seg Out 1000110
# PASS: Gray Input 1011 -> Internal Bin d -> Seg Out 0100001
```

